

CSAPP 3장: 컴퓨터는 코드를 어떻게 이해하는가

입문편 — 중3도 알아들을 수 있게 | 이거 먼저 읽고 심화편으로

먼저 이 챕터가 뭘 얘기인지부터

우리가 C언어로 코드를 짜면, 컴퓨터는 그걸 바로 이해 못 한다. 컴퓨터는 **0과 1(전기 신호)**밖에 모른다. 그래서 누군가가 우리 코드를 0과 1로 **번역**해줘야 한다.

그 번역기가 **컴파일러(compiler)**다.

이 챕터에서는 "그 번역이 어떻게 이루어지는지", 그리고 "번역된 결과물(기계어)을 읽는 법"을 배운다.

비유하면:

내가 한국어로 편지를 썼는데, 받는 사람은 영어만 안다고 하자.

C 코드 = 내가 쓴 한국어 편지

컴파일러 = 한→영 번역기

어셈블리 = 번역된 영어 편지 (사람도 읽을 수 있음)

기계어 = 그 영어를 바코드로 변환한 것 (컴퓨터만 읽을 수 있음)

3.1 x86의 역사 (가볍게만)

지금 우리가 쓰는 노트북/데스크톱 CPU의 대부분은 **x86-64**라는 설계를 따른다.

이건 Intel이라는 회사가 1978년에 만든 칩(8086)에서 시작해서 계속 업그레이드된 거다.

비유하면:

게임기로 치면 닌텐도 패미컴(1983) → 슈퍼패미컴 → N64 → Wii → Switch 이렇게 계속 발전한 것.

근데 중요한 건 **옛날 게임도 여전히 돌아간다**는 거다. 이걸 "하위 호환"이라고 한다.

이것만 기억해:

- 처음엔 **16비트**(1978, 8086) → **32비트**(1985, i386) → **64비트**(2004, x86-64)
- "비트 수"가 올라가면? → 한 번에 다룰 수 있는 데이터가 커지고, 쓸 수 있는 메모리도 커진다
- 32비트 = 최대 4GB 메모리, 64비트 = 사실상 무한대 메모리

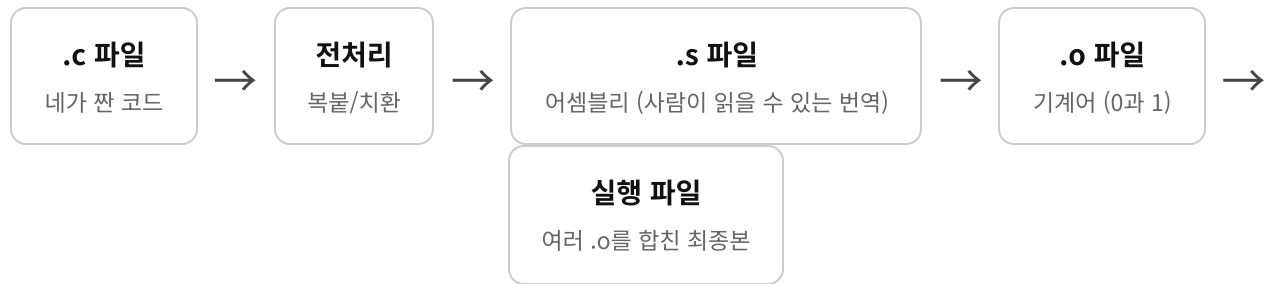
무어의 법칙

CPU 안의 트랜지스터(아주 작은 스위치) 수가 **약 1.5년마다 2배**로 늘어난다는 관찰. 1978년에 29,000개였던 게 2013년에 14억 개가 됐다. 이게 컴퓨터가 계속 빨라진 이유다.

3.2 코드가 실행 파일이 되는 과정

먼저 한 줄 요약

네가 쓴 C 코드(.c 파일)는 **4단계**를 거쳐서 컴퓨터가 실행할 수 있는 파일이 된다.



각 단계를 쉽게 설명

전체를 요리에 비유하면:

1단계 - 전처리: 레시피에 "엄마 레시피 참고"라고 써있으면 그 레시피를 가져와서 붙여넣는다. `#include` 가 이거다.

2단계 - 컴파일: 한국어 레시피를 "계란 깨라 → 소금 넣어라 → 저어라" 같은 아주 단순한 명령어로 바꾼다. 이게 어셈블리다.

3단계 - 어셈블: 그 단순한 명령어들을 기계가 읽을 수 있는 숫자(바코드)로 바꾼다. 이게 기계어다.

4단계 - 링크: 여러 재료(파일)를 하나로 합쳐서 완성된 요리(실행 파일)를 만든다.

터미널 명령어

실제로 이 과정을 직접 할 수 있다:

```
# 어셈블리까지만 만들기 (결과: mstore.s)
gcc -Og -S mstore.c

# 기계어까지 만들기 (결과: mstore.o)
gcc -Og -c mstore.c
```

```
# 실행 파일까지 전부 만들기 (결과: prog)
gcc -Og -o prog main.c mstore.c

# 기계어를 다시 어셈블리로 역번역해서 보기
objdump -d mstore.o
```

-Og가 뭔데?

"최적화 하지 마"라는 뜻이다. 컴파일러가 코드를 번역할 때 "더 빠르게"라고 멋대로 코드를 바꿔버리는데, -Og를 붙이면 원래 코드 구조를 최대한 유지해준다. 공부할 때 쓰는 옵션.

실제 예제

이 C 코드를:

```
void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);    // x*y 계산
    *dest = t;               // 결과를 dest에 저장
}
```

컴파일러가 이렇게 번역한다 (어셈블리):

```
multstore:
    pushq   %rbx             ← rbx를 잠깐 백업해둬
    movq    %rdx, %rbx       ← dest 주소를 rbx에 복사
    call    mult2            ← mult2 함수 호출
    movq    %rax, (%rbx)      ← 결과를 dest에 저장
    popq    %rbx             ← rbx 복원
    ret                     ← 끝, 돌아가
```

무슨 말인지 하나씩 뜯어보자:

어셈블리 읽기의 기본 3가지

1. **%rax** , **%rbx** 같은 것 = **레지스터**. CPU 안에 있는 아주 작은 저장 공간이다. 변수라고 생각하면 된다. 16개 있다.
2. **movq A, B** = "A에 있는 값을 B로 복사해라". q 는 8바이트짜리라는 뜻.
3. **(%rbx)** = "rbx에 적힌 주소로 가서 그 메모리를 봐라". C에서 ***ptr** 과 같은 뜻이다.

레지스터를 쉽게 이해하면:

계산기에 M+ 버튼 있잖아? 숫자를 잠깐 저장해두는 거. 레지스터가 그거다. CPU 안에 16개의 M+ 버튼이 있는 것. 메모리 (RAM)보다 훨씬 빠르지만 공간이 아주 작다.

CPU가 보는 세계

C에서는 "이건 int고, 저건 문자열이야"라고 타입을 구분하지만, CPU한테는 **그런 거 없다**. CPU가 아는 건 딱 4가지:

1. **프로그램 카운터 (%rip):** "지금 몇 번째 줄 실행 중이야?" 를 알려주는 표시기. 책 읽을 때 손가락으로 줄 짚는 거랑 같다.
2. **레지스터 16개:** CPU 안의 초고속 임시 저장 공간. 위에서 설명한 M+ 버튼.
3. **조건 플래그:** "방금 계산 결과가 0이었나? 음수였나?" 같은 정보. if문을 실행하려면 이게 필요하다.
4. **메모리:** 그냥 거대한 바이트 배열. CPU 입장에서는 int도 char*도 다 똑같이 "숫자 덩어리"다.

기계를 다시 어셈블리로 보기 (디스어셈블리)

```
linux> objdump -d mstore.o
```

```
0: 53          push    %rbx
1: 48 89 d3    mov     %rdx,%rbx
4: e8 00 00 00 00 callq   9
9: 48 89 03    mov     %rax,(%rbx)
c: 5b          pop     %rbx
d: c3          retq
```

읽는 법:

맨 왼쪽 = 위치(주소), 가운데 = 기계어(진짜 0과 1을 16진수로 표현), 오른쪽 = 어셈블리(사람이 읽을 수 있는 버전)

예를 들어 53 이라는 숫자 1개가 push %rbx 라는 명령어인 거다. 48 89 d3 이라는 숫자 3개가 mov %rdx,%rbx 라는 명령어.

명령어마다 길이가 다르다 (1바이트~15바이트). 자주 쓰는 명령어일수록 짧다.

3.3 데이터 크기 (Data Formats)

핵심 질문: 데이터 하나가 몇 바이트인가?

컴퓨터에서 모든 데이터는 바이트(byte) 단위로 저장된다. 1바이트 = 8비트 = 0과 1이 8개.

비유하면:

바이트는 "칸"이라고 생각해. 숫자가 작으면 1칸이면 되고, 큰 숫자는 여러 칸이 필요하다.

char = 1칸, short = 2칸, int = 4칸, long = 8칸.

C 타입	Intel이 부르는 이름	어셈블리 접미사	크기	쉽게
char	Byte	b	1바이트	글자 1개
short	Word	w	2바이트	작은 숫자
int	Double word	l	4바이트	보통 숫자
long	Quad word	q	8바이트	큰 숫자
포인터	Quad word	q	8바이트	주소
float	Single precision	s	4바이트	소수점 (보통)
double	Double precision	l	8바이트	소수점 (정밀)

왜 이름이 이상한가?

이건 역사 때문이다:

Intel이 처음 CPU를 만들었을 때(1978년) 16비트였다. 그때 "16비트 = 1 Word(단어)"라고 정했다.

그 뒤로 32비트가 나오자 → 16비트의 2배니까 = **Double word**

64비트가 나오자 → 16비트의 4배니까 = **Quad word**

다른 분야에서는 보통 "word = 32비트"인데 **Intel만 "word = 16비트"**다. 시험에 나올 수 있다!

어셈블리 접미사 (b, w, l, q)

어셈블리 명령어 뒤에 글자가 하나 붙는데, 이게 "몇 바이트짜리 데이터를 다루는 건지" 알려준다:

```

movb → 1바이트 복사 (byte)      예: char
movw → 2바이트 복사 (word)       예: short
movl → 4바이트 복사 (long word)  예: int
movq → 8바이트 복사 (quad word)  예: long, 포인터

```

외우는 법:

byte(1) → word(2) → long(4) → quad(8)

크기가 1 → 2 → 4 → 8로 2배씩 커진다!

여기까지 이해했는지 체크

아래 질문에 답할 수 있으면 3.1~3.3은 이해한 것이다:

1. C 코드가 실행 파일이 되려면 거치는 4단계는? → 전처리 → 컴파일 → 어셈블 → 링크
2. 어셈블리는 뭔가? → 기계어를 사람이 읽을 수 있게 적은 것
3. `gcc -S` 는 뭘 하나? → 어셈블리 파일(.s)까지만 만든다
4. 레지스터가 뭔가? → CPU 안의 초고속 저장 공간 (변수 같은 것)
5. x86에서 "word"는 몇 비트? → 16비트 (다른 데서는 보통 32비트)
6. `movq` 의 `q` 는 무슨 뜻? → quad word = 8바이트
7. `int`는 어셈블리에서 무슨 접미사? → `l` (4바이트)
8. 포인터는 x86-64에서 몇 바이트? → 8바이트
9. CPU는 `int`와 `char*`를 구분할 수 있나? → 못한다. 다 숫자 덩어리.
10. `%rip`은 뭔가? → 지금 실행 중인 명령어 위치를 가리키는 포인터

여기까지 읽었으면 → 심화편으로 넘어가기

심화편에서는 같은 내용을 좀 더 기술적인 용어와 디테일로 다시 본다.